DSA0108
OBJECT ORIENTED PROGRAMMING WITH C++

CO2-AT2-SCENARIO BASED ASSESMENT

NAME:E.RAGUL

REG NO:192571032

1. A hospital wants to estimate consultation charges for different patient categories. Design a C++ program for a Consultation Charge System. Design a C++ program for hospital management

Task:
A. Use function overloading to calculate charge for general consultation and specialist consultation.
b. Use call by reference to modify the final bill amount.
c. Use default arguments to add a standard service charge if needed.
d. Use function prototyping for all calculation functions.
e. Display the patient type and total consultation charge.

ANSWER:

Program:

```
#include <iostream>

#include <string>

using namespace std;

double calculateCharge(double consultationFee);

double calculateCharge(double consultationFee, double specialistFee);

void updateBill(double &bill);

double addServiceCharge(double bill, double serviceCharge = 100);

int main()

{

   int choice;

   string patientType;

   double fee, specialistFee, bill;

   cout << "1. General Consultation\n";

   cout << "2. Specialist Consultation\n";

   cout << "Enter Choice: ";

   cin >> choice;

   if(choice==1)
```

```
{
    patientType="General";
    cout<<"Enter Consultation Fee: ";
    cin>>fee;
    bill=calculateCharge(fee);
  }
  else
  {
    patientType="Specialist";
    cout<<"Enter Consultation Fee: ";
    cin>>fee;
    cout<<"Enter Specialist Fee: ";
    cin>>specialistFee;
    bill=calculateCharge(fee,specialistFee);
  }
  bill=addServiceCharge(bill);
  updateBill(bill);
  cout<<"\nPatient Type : "<<patientType;
  cout<<"\nTotal Consultation Charge : "<<bill;
  return 0;
}
double calculateCharge(double consultationFee)
{
  return consultationFee;
}
double calculateCharge(double consultationFee,double specialistFee)
{
  return consultationFee+specialistFee;
```

}

void updateBill(double &bill)

{

bill=bill-50;

}

double addServiceCharge(double bill,double serviceCharge)

{

return bill+serviceCharge;

}

SCREENSHORT

INPUT:

Run | Debug | Stop | Share | Saved | {} Beautify

main.cpp

```
#include <iostream>
#include <string>
using namespace std;
double calculateCharge(double consultationFee);
double calculateCharge(double consultationFee, double specialistFee);
void updateBill(double &bill);
double addServiceCharge(double bill, double serviceCharge = 100);

int main()
{
    int choice;
    string patientType;
    double fee, specialistFee, bill;

    cout << "1. General Consultation\n";
    cout << "2. Specialist Consultation\n";
    cout << "Enter Choice: ";
    cin >> choice;

    if(choice==1)
    {
        patientType="General";
        cout<<"Enter Consultation Fee: ";
        cin>>fee;

        bill=calculateCharge(fee);
    }
    else
    {
        patientType="Specialist";
        cout<<"Enter Consultation Fee: ";
        cin>>fee;
        cout<<"Enter Specialist Fee: ";
```

Run | Debug | Stop | Share | Saved | {} Beautify

main.cpp

```
34          cin>>specialistFee;
35
36          bill=calculateCharge(fee,specialistFee);
37      }
38
39      bill=addServiceCharge(bill);
40
41      updateBill(bill);
42
43      cout<<"\nPatient Type : "<<patientType;
44      cout<<"\nTotal Consultation Charge : "<<bill;
45
46      return 0;
47  }
48
49  double calculateCharge(double consultationFee)
50  {
51      return consultationFee;
52  }
53
54  double calculateCharge(double consultationFee,double specialistFee)
55  {
56      return consultationFee+specialistFee;
57  }
58
59  void updateBill(double &bill)
60  {
61      bill=bill-50;
62  }
63
64  double addServiceCharge(double bill,double serviceCharge)
65  {
66      return bill+serviceCharge;
67  }
```

OUTPUT:

```
1. General Consultation
2. Specialist Consultation
Enter Choice: 2
Enter Consultation Fee: 500
Enter Specialist Fee: 700

Patient Type : Specialist
Total Consultation Charge : 1250
```

2. Design a C++ program Fitness Centre Membership System. A gym wants to manage member subscriptions, track plans, and calculate membership fees.

Task:
Create a class Member with attributes: Member ID, Name, Plan Type, and Fee.
Implement member functions to assign plans, calculate fees, and display membership details.
Use an array of objects to manage multiple members.
Add validation for valid membership plans.
Include a static member to track total active members.
Evaluate how your design supports easy modification of pricing plans.

ANSWER

PROGRAM:

```
#include<iostream>
#include<string>
using namespace std;
class Member
{
private:
   int memberID;
   string name;
   string planType;
   double fee;
public:
   static int totalMembers;
   void assignPlan()
   {
      cout<<"Enter Member ID: ";
      cin>>memberID;
      cin.ignore();
      cout<<"Enter Name: ";
      getline(cin,name);
```

```
        cout<<"Enter Plan (Basic/Premium/VIP): ";
        getline(cin,planType);

        if(planType=="Basic")
            fee=1000;
        else if(planType=="Premium")
            fee=2000;
        else if(planType=="VIP")
            fee=3000;
        else
        {
            cout<<"Invalid Plan\n";
            planType="Invalid";
            fee=0;
        }
        if(fee>0)
            totalMembers++;
    }
    void display()
    {
        cout<<"\nMember ID : "<<memberID;
        cout<<"\nName : "<<name;
        cout<<"\nPlan : "<<planType;
        cout<<"\nFee : "<<fee<<"\n";
    }
    static void displayTotal()
    {
```

```
        cout<<"\nTotal Active Members : "<<totalMembers;
    }
};
int Member::totalMembers=0;
int main()
{
    int n;
    cout<<"Enter Number of Members: ";
    cin>>n;
    Member m[50];
    for(int i=0;i<n;i++)
    {
        cout<<"\nMember "<<i+1<<endl;
        m[i].assignPlan();
    }
    cout<<"\nMembership Details\n";
    for(int i=0;i<n;i++)
    {
        m[i].display();
    }
    Member::displayTotal();
    cout<<"\n\nPricing plans can be modified easily by changing fee values inside assignPlan().";
    return 0;
}
```

SCREENSHORT

INPUT:

Run | Debug | Stop | Share | Saved | {} Beautify

main.cpp

```
53
54      static void displayTotal()
55      {
56          cout<<"\nTotal Active Members : "<<totalMembers;
57      }
58  };
59
60  int Member::totalMembers=0;
61
62  int main()
63  {
64      int n;
65
66      cout<<"Enter Number of Members: ";
67      cin>>n;
68
69      Member m[50];
70
71      for(int i=0;i<n;i++)
72      {
73          cout<<"\nMember "<<i+1<<endl;
74          m[i].assignPlan();
75      }
76
77      cout<<"\nMembership Details\n";
78
79      for(int i=0;i<n;i++)
80      {
81          m[i].display();
82      }
83
84      Member::displayTotal();
85
86      cout<<"\n\nPricing plans can be modified easily by changing fee values inside assignPlan().";
87
88      return 0;
89  }
```

OUTPUT:

input

```
Enter Number of Members: 2

Member 1
Enter Member ID: 143
Enter Name: ragul
Enter Plan (Basic/Premium/VIP): Basic

Member 2
Enter Member ID: 420
Enter Name: rag
Enter Plan (Basic/Premium/VIP): vip
Invalid Plan

Membership Details

Member ID : 143
Name : ragul
Plan : Basic
Fee : 1000

Member ID : 420
Name : rag
Plan : Invalid
Fee : 0

Total Active Members : 1

Pricing plans can be modified easily by changing fee values inside assignPlan().
```

3. A bank wants to calculate loan repayment details for customers. Design a C++ program for a Loan Repayment System.

Task:
A. Use function prototyping to declare EMI-related functions.
b. Use default arguments for default interest rate where applicable.
c. Apply call by reference to update payable amount after subsidy.
d. Use inline functions for simple interest calculations.
e. Display the loan amount, interest, and repayment details clearly.

ANSWER
PROGRAM:

```
#include<iostream>

using namespace std;

double calculateInterest(double loan,double rate=8);

void applySubsidy(double &amount);

double calculateEMI(double amount,int months);

inline double simpleInterest(double p,double r)

{

   return (p*r)/100;

}

int main()

{

   double loan,interest,total,emi;

   int months;

   cout<<"Enter Loan Amount: ";

   cin>>loan;

   cout<<"Enter Months: ";

   cin>>months;
```

```
    interest=calculateInterest(loan);
    total=loan+interest;
    applySubsidy(total);
    emi=calculateEMI(total,months);
    cout<<"\nLoan Amount : "<<loan;
    cout<<"\nInterest : "<<interest;
    cout<<"\nTotal Payable : "<<total;
    cout<<"\nMonthly EMI : "<<emi;
    return 0;
}
double calculateInterest(double loan,double rate)
{
    return simpleInterest(loan,rate);
}
void applySubsidy(double &amount)
{
    amount=amount-500;
}
double calculateEMI(double amount,int months)
{
    return amount/months;
}
```

SCREENSHORT
INPUT:

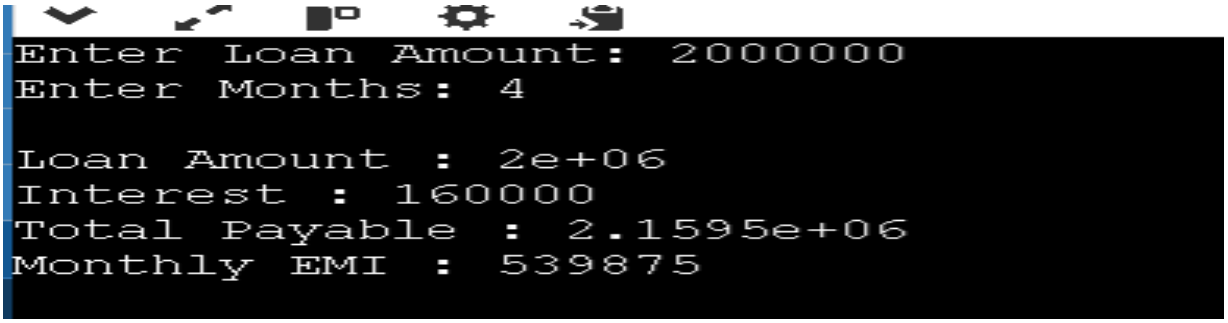
Run | Debug | Stop | Share | Saved | {} Beautify

main.cpp

```cpp
#include<iostream>
using namespace std;

// Function Prototypes
double calculateInterest(double loan,double rate=8);
void applySubsidy(double &amount);
double calculateEMI(double amount,int months);

inline double simpleInterest(double p,double r)
{
    return (p*r)/100;
}

int main()
{
    double loan,interest,total,emi;
    int months;

    cout<<"Enter Loan Amount: ";
    cin>>loan;

    cout<<"Enter Months: ";
    cin>>months;

    interest=calculateInterest(loan);

    total=loan+interest;

    applySubsidy(total);

    emi=calculateEMI(total,months);

    cout<<"\nLoan Amount : "<<loan;
    cout<<"\nInterest : "<<interest;
    cout<<"\nTotal Payable : "<<total;
    cout<<"\nMonthly EMI : "<<emi;

    return 0;
```

```cpp
double calculateInterest(double loan,double rate)
{
    return simpleInterest(loan,rate);
}

void applySubsidy(double &amount)
{
    amount=amount-500;
}

double calculateEMI(double amount,int months)
{
    return amount/months;
}
```

OUTPUT:



4. A delivery company wants to automate the calculation of parcel delivery charges based on parcel weight, delivery distance, and delivery type. Design and implement a C++ program using functions and object-oriented programming concepts to develop a Parcel Charge Management System.

a. Create a class Parcel with appropriate private data members such as parcel type, weight, distance, and packaging charge.
b. Use public member functions to read parcel details, calculate the delivery charge, and display the parcel information.
c. Use function overloading to calculate charges for Normal Delivery and Express Delivery.
d. Use default arguments to assign a default packaging charge when it is not specified by the user.
e. Use an inline function to calculate the base charge based on parcel weight and delivery distance.
f. Maintain the total number of parcels processed using a static data member and display it through a static member function.
ANSWER
PROGRAM:

```
#include<iostream>

#include<string>

using namespace std;

class Parcel

{

private:

    string type;

    double weight;
```

```
    double distance;
    double packageCharge;
    static int totalParcels;
public:
    void readDetails(double p=50)
    {
        cout<<"Enter Parcel Type: ";
        cin>>type;
        cout<<"Enter Weight: ";
        cin>>weight;
        cout<<"Enter Distance: ";
        cin>>distance;
        packageCharge=p;
        totalParcels++;
    }
    inline double baseCharge()
    {
        return weight*10+distance*2;
    }
    double calculateCharge()
    {
        return baseCharge()+packageCharge;
    }
```

```
    double calculateCharge(int express)
    {
        return baseCharge()+packageCharge+200;
    }
    void display(int express=0)
    {
        double total;
        if(express==0)
            total=calculateCharge();
        else
            total=calculateCharge(express);
        cout<<"\nParcel Type : "<<type;
        cout<<"\nWeight : "<<weight;
        cout<<"\nDistance : "<<distance;
        cout<<"\nTotal Charge : "<<total<<"\n";
    }
    static void showTotal()
    {
        cout<<"\nTotal Parcels Processed : "<<totalParcels;
    }
};
```

```
int Parcel::totalParcels=0;

int main()

{

   Parcel p1,p2;

   p1.readDetails();

   p1.display();

   p2.readDetails();

   p2.display(1);

   Parcel::showTotal();

   return 0;

}
```

SCREENSHORT

INPUT:

main.cpp

```
47
48      void display(int express=0)
49      {
50          double total;
51
52          if(express==0)
53              total=calculateCharge();
54          else
55              total=calculateCharge(express);
56
57          cout<<"\nParcel Type : "<<type;
58          cout<<"\nWeight : "<<weight;
59          cout<<"\nDistance : "<<distance;
60          cout<<"\nTotal Charge : "<<total<<"\n";
61      }
62
63      static void showTotal()
64      {
65          cout<<"\nTotal Parcels Processed : "<<totalParcels;
66      }
67  };
68
69  int Parcel::totalParcels=0;
70
71  int main()
72  {
73      Parcel p1,p2;
74
75      p1.readDetails();
76      p1.display();
77
78      p2.readDetails();
79      p2.display(1);
80
81      Parcel::showTotal();
82
83      return 0;
84  }
```

OUTPUT:

```
Enter Parcel Type: 2
Enter Weight: 200kg
Enter Distance:
Parcel Type : 2
Weight : 200
Distance : 0
Total Charge : 2050
Enter Parcel Type: Enter Weight: Enter Distance:
Parcel Type :
Weight : 6.33317e-310
Distance : 6.33317e-310
Total Charge : 250

Total Parcels Processed : 2
```

5. A utility company wants to automate the calculation of water bills for domestic and commercial consumers. Design and implement a C++ program using functions and object-oriented programming concepts to develop a Water Billing System. create a class Consumer with appropriate private data members such as consumer ID, consumer type, water consumption (in liters), and bill amount.
b. Use public member functions to input consumer details, calculate the water bill, and display the billing information.
c. Implement function overloading to calculate water bills for Domestic and Commercial consumers using different tariff rates.
d. Use call by reference to update the bill amount after applying a government subsidy.
e. Use default arguments to include a service tax, where a default tax rate is applied unless specified.
f. Declare all functions using function prototyping before their definitions.
g. Maintain the total number of consumers processed using a static data member and display it through a static member function.
h. Create an array of objects to store and process the details of multiple consumers, and display the consumer type, water consumption, subsidy-adjusted bill amount

ANSWER
PROGRAM:

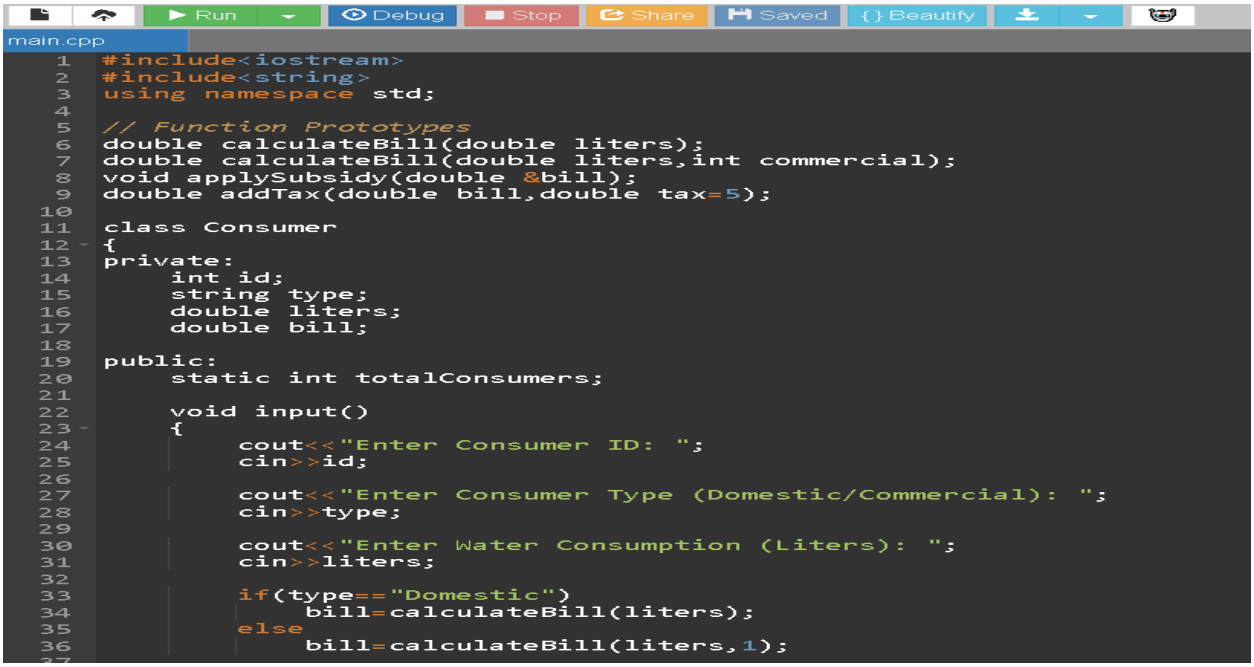
```
#include<iostream>

#include<string>

using namespace std;

double calculateBill(double liters);

double calculateBill(double liters,int commercial);

void applySubsidy(double &bill);

double addTax(double bill,double tax=5);

class Consumer

{

private:

   int id;

   string type;

   double liters;
```

```
    double bill;
public:
    static int totalConsumers;
    void input()
    {
        cout<<"Enter Consumer ID: ";
        cin>>id;
        cout<<"Enter Consumer Type (Domestic/Commercial): ";
        cin>>type;
        cout<<"Enter Water Consumption (Liters): ";
        cin>>liters;
        if(type=="Domestic")
            bill=calculateBill(liters);
        else
            bill=calculateBill(liters,1);
        bill=addTax(bill);
        applySubsidy(bill);
        totalConsumers++;
    }
    void display()
    {
        cout<<"\nConsumer ID : "<<id;
        cout<<"\nConsumer Type : "<<type;
```

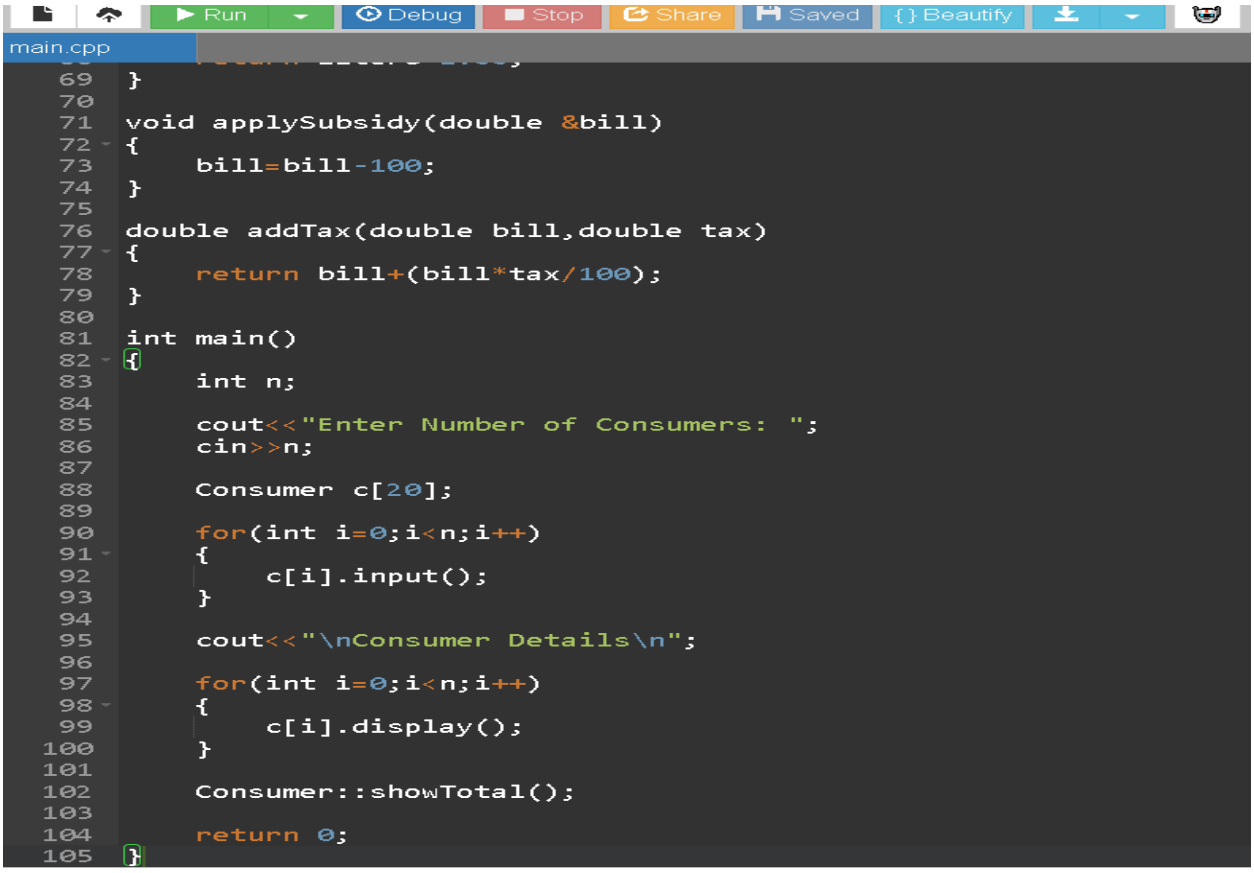
```
        cout<<"\nWater Consumption : "<<liters<<" Liters";
        cout<<"\nBill Amount : "<<bill<<"\n";
    }
    static void showTotal()
    {
        cout<<"\nTotal Consumers : "<<totalConsumers;
    }
};
int Consumer::totalConsumers=0;
double calculateBill(double liters)
{
    return liters*0.50;
}
double calculateBill(double liters,int commercial)
{
    return liters*1.00;
}
void applySubsidy(double &bill)
{
    bill=bill-100;
}
double addTax(double bill,double tax)
{
```

```
    return bill+(bill*tax/100);
}
int main()
{
    int n;
    cout<<"Enter Number of Consumers: ";
    cin>>n;

    Consumer c[20];
    for(int i=0;i<n;i++)
    {
        c[i].input();
    }
    cout<<"\nConsumer Details\n";
    for(int i=0;i<n;i++)
    {
        c[i].display();
    }
    Consumer::showTotal();
    return 0;
}
```

SCREENSHORT
INPUT:

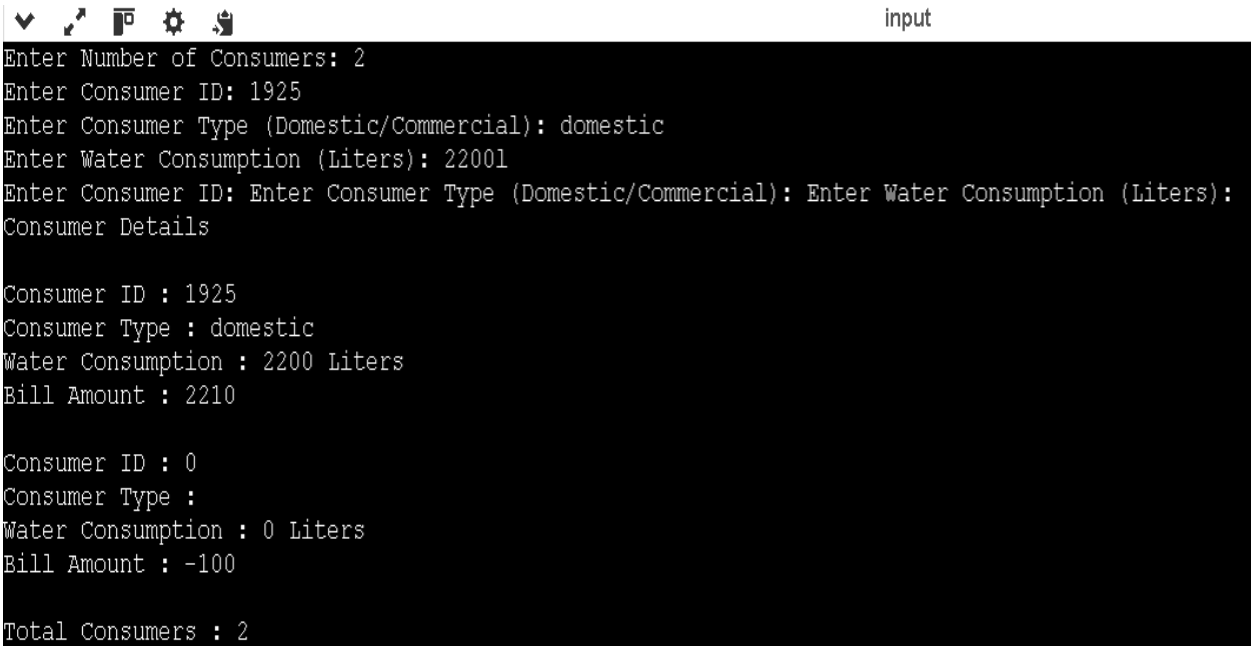



```cpp
#include<iostream>
#include<string>
using namespace std;

// Function Prototypes
double calculateBill(double liters);
double calculateBill(double liters,int commercial);
void applySubsidy(double &bill);
double addTax(double bill,double tax=5);

class Consumer
{
private:
    int id;
    string type;
    double liters;
    double bill;

public:
    static int totalConsumers;

    void input()
    {
        cout<<"Enter Consumer ID: ";
        cin>>id;

        cout<<"Enter Consumer Type (Domestic/Commercial): ";
        cin>>type;

        cout<<"Enter Water Consumption (Liters): ";
        cin>>liters;

        if(type=="Domestic")
            bill=calculateBill(liters);
        else
            bill=calculateBill(liters,1);
```




```cpp
}

void applySubsidy(double &bill)
{
    bill=bill-100;
}

double addTax(double bill,double tax)
{
    return bill+(bill*tax/100);
}

int main()
{
    int n;

    cout<<"Enter Number of Consumers: ";
    cin>>n;

    Consumer c[20];

    for(int i=0;i<n;i++)
    {
        c[i].input();
    }

    cout<<"\nConsumer Details\n";

    for(int i=0;i<n;i++)
    {
        c[i].display();
    }

    Consumer::showTotal();

    return 0;
}
```

OUTPUT:


input
Enter Number of Consumers: 2
Enter Consumer ID: 1925
Enter Consumer Type (Domestic/Commercial): domestic
Enter Water Consumption (Liters): 2200l
Enter Consumer ID: Enter Consumer Type (Domestic/Commercial): Enter Water Consumption (Liters):
Consumer Details
Consumer ID : 1925
Consumer Type : domestic
Water Consumption : 2200 Liters
Bill Amount : 2210
Consumer ID : 0
Consumer Type :
Water Consumption : 0 Liters
Bill Amount : -100
Total Consumers : 2